JAVA FULL STACK BACKEND COMPLETE STUDY NOTES

Based on Telusko's 63-Hour Complete Java Development Course by Navin Reddy

TABLE OF CONTENTS

6.1 Service Discovery, Gateway, Resilience

Module 7: Design Patterns
7.1 Creational, Structural, Behavioral Patterns

Cross-Concept Comparison Tables
End-of-Module Mock Interview (30 Questions)
Final Cheat Sheet

MODULE 1: JAVA FUNDAMENTALS

1.1 Introduction to Java

Java is a high-level, class-based, OOP language developed by James Gosling at Sun Microsystems (now Oracle) in 1995. It follows "Write Once, Run Anywhere" (WORA) — compiled bytecode runs on any platform with a JVM.

Key Features

• Platform Independence: .java → javac → .class (bytecode) → JVM executes. Bytecode is platform-independent; JVM is platform-specific.

• Object-Oriented: Everything except primitives is an object. Supports inheritance, polymorphism, encapsulation, abstraction.

• Automatic Memory Management: Garbage Collector automatically reclaims unreferenced object memory.

• Rich Standard Library: Collections, I/O, networking, JDBC, security, concurrency.

• Strong Type System: Statically typed — compiler catches type mismatches at compile time.

• Security: JVM sandbox, bytecode verifier, security manager, cryptography APIs.

■ Additional Insight (beyond video): Java 8 introduced lambdas/streams. Java 9 added modules. Current LTS: Java 11, 17, 21. For production, use Java 17+.

JDK vs JRE vs JVM

• JVM: Executes bytecode, manages memory (stack, heap, method area), GC, JIT compilation.

• JRE: JVM + core libraries. Needed to RUN Java programs.

• JDK: JRE + development tools (javac, jdb, javadoc). Needed to DEVELOP.

■■ JDK = JRE + Dev Tools. JRE = JVM + Libraries.

JVM Architecture

Class Loader Subsystem Loading → Linking → Initialization		
	Execution Engine	
	Interpreter | JIT | GC	
Native OS (Windows/Linux/macOS)		

Hello World

```
public class HelloWorld {
public static void main(String[] args) {
System.out.println("Hello, Java!");
}
}
```

• public class HelloWorld — class must be public, filename matches class name.

• public static void main(String[] args) — entry point. static = no instance needed. String[] args = CLI arguments.

• System.out.println() — prints to console with newline.

```
javac HelloWorld.java # .java -> .class
java HelloWorld # JVM loads & executes
```

1.2 Data Types

```
byte b=127; short s=32767; int i=2147483647; long l=999L;
float f=3.14f; double d=3.14159; char c='A'; boolean flag=true;
```

- Widening (implicit): byte->short->int->long->float->double
- Narrowing (explicit): requires cast: (int)3.14. Potential data loss.

1.3 Operators & Control Flow

```
Arithmetic: + - * / %
Comparison: == != > < >= <=
Logical: && || !
Ternary: x > 0 ? "pos" : "neg"
```

```
if (score >= 90) { grade='A'; } else if (score >= 80) { grade='B'; } else { grade='F'; }

switch (day) {
case 1: System.out.println("Mon"); break;
default: System.out.println("?");
}

for (int i=0; i<5; i++) { }
for (String n : names) { }
while (c < 10) { c++; }
do { c++; } while (c < 10);
```

1.4 Arrays & Strings

```
int[] nums = {1,2,3}; String[] names = new String[3];
int[][] mat = {{1,2},{3,4}};
int len = nums.length; // PROPERTY

String s1 = "Hello"; // pool
String s2 = new String("Hello"); // heap
s1.equals(s2); // true - compare value
s1 == s2; // false - compare reference
```

■■ Strings are immutable. Use StringBuilder for mutable strings.

1.5 I/O

```
Scanner sc = new Scanner(System.in);
String name = sc.nextLine();
int age = sc.nextInt();
sc.close();

// try-with-resources (Java 7+)
try (BufferedReader br = new BufferedReader(new FileReader("f.txt"))) {
String line;
while ((line = br.readLine()) != null) System.out.println(line);
} catch (IOException e) { e.printStackTrace(); }
```

Interview Questions

Q1: JDK vs JRE vs JVM?

Answer: JDK = JRE + dev tools. JRE = JVM + libs. JVM executes bytecode.

Q2: Why main() is public static void?

Answer: public=accessible; static=no instance needed; void=no return; JVM looks for exact signature.

Q3: 8 primitives?

Answer: byte, short, int, long, float, double, char, boolean.

Q4: Stack vs Heap?

Answer: Stack: per-thread, local vars/refs, ~1MB, auto. Heap: shared, objects, GC-managed.

Q5: Garbage Collection?

Answer: Auto reclaims unreachable objects. Generations: Young (Eden/S0/S1) -> Old -> Metaspace.

Q6: String vs StringBuilder vs StringBuffer?

Answer: String: immutable. StringBuilder: mutable, not thread-safe. StringBuffer: mutable, thread-safe.

Q7: == vs .equals()?

Answer: == compares references. .equals() compares values.

Q8: Can we run Java without main()?

Answer: No (Java 7+). JVM requires main method as entry point.

Q9: Autoboxing?

Answer: Auto int->Integer conversion by compiler. Reverse = unboxing.

Q10: Checked vs Unchecked?

Answer: Checked: must handle/declare (IOException). Unchecked: RuntimeException, not required.

Q11: ArrayList vs LinkedList?

Answer: ArrayList: O(1) get, O(n) insert middle. LinkedList: O(n) get, O(1) insert middle.

Q12: HashMap vs ConcurrentHashMap?

Answer: HashMap: not thread-safe, allows null. ConcurrentHashMap: thread-safe, no null, lock striping.

Q13: equals() and hashCode() contract?

Answer: If equals() true, hashCode MUST be equal. Override both for HashMap/HashSet.

Q14: static keyword?

Answer: Class-level member. Shared across instances. Called without object.

Q15: final keyword?

Answer: final variable=constant; final method=no override; final class=no extension.

Q16: What is try-with-resources?

Answer: Java 7+. Auto-closes AutoCloseable resources. No finally needed for close.

Q17: Interface vs Abstract Class?

Answer: Interface: contract, multiple inheritance, all methods abstract+default+static. Abstract: state+behavior, single inheritance.

Q18: What is functional interface?

Answer: Single abstract method. Used with lambdas. @FunctionalInterface. Examples: Runnable, Predicate.

Q19: lambda syntax?

Answer: (params) -> expression or { statements }. Implements functional interface.

Q20: Stream intermediate vs terminal?

Answer: Intermediate: lazy, returns Stream (filter/map/sorted). Terminal: triggers execution (collect/forEach/reduce).

Quick Revision

- JVM executes bytecode; JRE=JVM+libs; JDK=JRE+tools
- 8 primitives: byte,short,int,long,float,double,char,boolean
- main(): public static void main(String[])
- Stack=local vars; Heap=objects (GC-managed)

- String immutable; StringBuilder mutable
- == compares refs; .equals() compares values
- Checked exceptions must be handled
- Generics = compile-time type safety

MODULE 2: OBJECT-ORIENTED PROGRAMMING

2.1 Classes & Objects

Class = blueprint. Object = instance with state (fields) and behavior (methods).

```
public class Student {
private String name; // state
public Student(String n) { this.name = n; } // constructor
public void study() { System.out.println(name+" studying"); } // behavior
}
Student s = new Student("John"); s.study();
```

- Constructor: name=className, no return type, called via new.
- Default no-arg constructor provided only if NO constructor defined.
- this() chains constructors; super() calls parent constructor. Both must be first statement.

2.2 Encapsulation

Bundle data+methods, hide internal state via private fields, expose via public methods.

Modifier	Class	Package	Subclass	World
private	Y	N	N	N
default	Y	Y	N	N
protected	Y	Y	Y	N
public	Y	Y	Y	Y

2.3 Inheritance

extends = subclass inherits from superclass. Single inheritance only.

```
class Animal { protected String name; void eat() { } }
class Dog extends Animal { void bark() { } }
// Dog d = new Dog(); d.eat(); d.bark();
```

- super() calls parent constructor; super.method() calls parent method.
- Override rules: same signature, covariant return, cannot be more restrictive. @Override annotation recommended.

2.4 Polymorphism

Overloading (compile-time)

```
int add(int a,int b) { } double add(double a,double b) { } int add(int a,int b,int c) { }
```

Overriding (runtime)

```
Animal a = new Dog(); a.eat(); // Dog's eat() called at runtime
```

■ Additional Insight (beyond video): Upcasting (Dog->Animal): safe/automatic. Downcasting (Animal->Dog): explicit, use instanceof check.

2.5 Abstraction

Abstract Class

```
abstract class Vehicle {
abstract void start(); // no body
void stop() { } // concrete
}
class Car extends Vehicle { void start() { } }
```

Interface

```
interface Drawable {
void draw();
default void print() { } // Java 8+
static void info() { } // Java 8+
}
class Circle implements Drawable { public void draw() { } }
```

Feature	Abstract Class	Interface
Has constructors?	Yes	No
Multiple inheritance?	No (single extends)	Yes (implements multiple)
Fields	Any (private/public)	public static final only
Methods	abstract+concrete	abstract+default+static
When to use	Shared base with state	Contract/capability

2.6 Static & Final

```
static int count; // shared across all instances
static void method() { } // class-level, no instance access
final int MAX = 100; // constant
final void cannotOverride() { } // cannot override
final class CannotExtend { } // cannot extend (String, Math)
```

2.7 Inner Classes

- Member Inner Class: at class level, can access outer private members.
- Static Nested: static, cannot access instance members.
- Anonymous: no name, used with method override. Common with Runnable.

```
Runnable r = new Runnable() {
@Override public void run() { }
};
new Thread(r).start();
```

Quick Revision

- Class=blueprint; Object=instance in heap
- Encapsulation: private fields + public getters/setters
- Inheritance: extends (single); Interface: implements (multi)
- Polymorphism: overloading(compile), overriding(runtime)
- Abstract class = state+behavior; Interface = contract
- static=class-level; final=cant change/extend/override
- Upcasting automatic; downcasting needs instanceof
- Override equals()+hashCode() together

MODULE 3: ADVANCED JAVA

3.1 Exception Handling

```
Throwable
Error (unrecoverable)
Exception
RuntimeException (unchecked: NPE, Arithmetic)
Checked (IOException, SQLException)
```

```
try { FileReader fr = new FileReader("f.txt"); }
catch (FileNotFoundException e) { System.out.println(e.getMessage()); }
catch (IOException e) { e.printStackTrace(); }
finally { System.out.println("Always runs"); }
```

- Multi-catch (Java 7+): catch (IOException | SQLException e)
- try-with-resources (Java 7+): auto-closes AutoCloseable.

```
public class InsufficientFundsException extends Exception {
public InsufficientFundsException(double amt) { super("Need "+amt); }
}
// throw new InsufficientFundsException(500);
```

3.2 Collections Framework

```
List: ArrayList, LinkedList
Set: HashSet, TreeSet, LinkedHashSet
Map: HashMap, TreeMap, LinkedHashMap
Queue: PriorityQueue, LinkedList
```

Feature	ArrayList	LinkedList
Internal	Resizable array	Doubly-linked
get(i)	O(1)	O(n)
insert middle	O(n) shift	O(1)

```
List l = new ArrayList<>(); l.add("A"); l.get(0);
Map m = new HashMap<>(); m.put("John",85);
Set s = new HashSet<>(); s.add(1); s.add(1); // 2nd ignored
```

3.3 Generics

```
class Box { T content; void set(T t) { } T get() { } }
Box b = new Box<>(); // diamond

// Wildcards:
List r = new ArrayList(); // read-only
List w = new ArrayList(); // write OK
```

3.4 Lambda & Functional Interfaces

```
@FunctionalInterface interface Calc { int op(int a,int b); }
Calc add = (a,b) -> a+b;
add.op(5,3); // 8

Predicate isEven = n -> n%2==0;
Function strLen = s -> s.length();
Consumer print = System.out::println;
Supplier rand = () -> Math.random();
```

3.5 Streams API

```
List nums = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
List evensSq = nums.stream()
.filter(n -> n%2==0) // lazy intermediate
.map(n -> n*n) // lazy intermediate
.collect(Collectors.toList()); // eager terminal

int sum = nums.stream().reduce(0, (a,b)->a+b);
nums.parallelStream().filter(n->n>5).forEach(System.out::println);
```

3.6 Multithreading

```
// 3 ways:
class MyThread extends Thread { public void run() { } }
class MyTask implements Runnable { public void run() { } }
new Thread(() -> System.out.println("Lambda")).start();

// Synchronized
public synchronized void inc() { count++; }
synchronized(this) { balance += amt; }

// ExecutorService
ExecutorService exec = Executors.newFixedThreadPool(3);
Future f = exec.submit(() -> { Thread.sleep(1000); return "OK"; });
String res = f.get(); exec.shutdown();
```

Quick Revision

- Checked exceptions must be handled; unchecked dont
- ArrayList: O(1) get; LinkedList: O(1) insert middle
- HashMap: O(1) avg, uses hashCode+equals
- Generics: compile-time type safety
- Lambdas: implement functional interfaces
- Streams: lazy intermediate, eager terminal
- Runnable: void run(); Callable: V call() throws
- volatile=visibility; synchronized=atomicity+visibility

MODULE 4: SPRING FRAMEWORK CORE

4.1 Dependency Injection

IoC (Inversion of Control): container manages object lifecycle. DI: dependencies injected by container.

Constructor Injection (Recommended)

```
@Service
public class UserService {
private final UserRepository repo;
public UserService(UserRepository repo) { this.repo = repo; }
}
```

Field Injection (Not Recommended)

```
@Service
public class OrderService {
@Autowired private OrderRepository repo;
}
```

■■ Field injection hides dependencies, makes testing harder. Prefer constructor injection.

4.2 Beans & Scopes

Scope	Behavior	Use Case
Singleton	One per container	Stateless beans
Prototype	New per request	Stateful beans
Request	One per HTTP req	Web data
Session	One per session	User session

```
@Component
@Scope("singleton") // default
public class MyService { }

@Bean
public UserService userService(UserRepository repo) {
return new UserService(repo);
}

@PostConstruct void init() { }
@PreDestroy void cleanup() { }
```

4.3 AOP

```
@Aspect @Component
public class LoggingAspect {
@Before("execution(* com.example.service.*.*(..))")
public void logBefore() { System.out.println("Before"); }

@Around("execution(* com.example.service.*.*(..))")
public Object logAround(ProceedingJoinPoint pjp) throws Throwable {
long start = System.currentTimeMillis();
Object result = pjp.proceed();
long time = System.currentTimeMillis()-start;
System.out.println("Took "+time+"ms");
return result;
}
}
```

4.4 Transaction Management

```
@Service
public class UserService {
@Transactional
public void createUser(User u) {
repo.save(u);
// if exception, rollback
}
@Transactional(readOnly=true)
public User get(Long id) { return repo.findById(id).orElseThrow(); }
}
```

Quick Revision

- IoC: container manages lifecycle; DI: injects dependencies
- Constructor injection preferred over field injection
- @Component stereotypes: @Service, @Repository, @Controller
- Scopes: Singleton(default), Prototype, Request, Session
- AOP: @Before, @After, @Around, @AfterReturning, @AfterThrowing
- @Transactional: declarative tx management
- Propagation: REQUIRED, REQUIRES_NEW, NESTED
- @PostConstruct / @PreDestroy: lifecycle hooks

MODULE 5: SPRING BOOT

5.1 Introduction

Spring Boot simplifies Spring development with auto-configuration, embedded servers, and production-ready features.

- Auto-Configuration: automatically configures beans based on classpath dependencies.
- Embedded Servers: Tomcat, Jetty, Undertow. No external deployment needed.
- Production-Ready: health checks, metrics, externalized configuration, Actuator.

```
@SpringBootApplication // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class Application {
public static void main(String[] args) {
SpringApplication.run(Application.class, args);
}
}
```

5.2 REST APIs

```
@RestController
@RequestMapping("/api/users")
public class UserController {
@GetMapping
public List getAll() { return service.findAll(); }

@GetMapping("/{id}")
public User get(@PathVariable Long id) { return service.findById(id); }

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public User create(@Valid @RequestBody User user) { return service.save(user); }

@PutMapping("/{id}")
public User update(@PathVariable Long id, @RequestBody User u) { return service.update(id,u); }

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) { service.delete(id); }
}
```

Exception Handling

```
@ControllerAdvice
public class GlobalExceptionHandler {
@ExceptionHandler(ProductNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ResponseEntity handleNotFound(ProductNotFoundException ex) {
return ResponseEntity.status(404).body(Map.of("error", ex.getMessage()));
}
}
```

5.3 Database with JPA

```
@Entity
@Table(name="products")
public class Product {
@Id @GeneratedValue(strategy=GenerationType.IDENTITY)
private Long id;
@NotBlank @Column(nullable=false)
```

```
private String name;
@NotNull @DecimalMin("0.0")
private BigDecimal price;
}

@Repository
public interface ProductRepository extends JpaRepository {
List findByNameContaining(String name);
@Query("SELECT p FROM Product p WHERE p.price > :min")
List findByPriceGreaterThan(@Param("min") BigDecimal min);
}

// application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

5.4 Security

```
@Configuration @EnableWebSecurity
public class SecurityConfig {
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
http.authorizeHttpRequests(auth -> auth
.requestMatchers("/api/public/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
).httpBasic();
return http.build();
}
@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

Quick Revision

- @SpringBootApplication = @Config + @EnableAutoConfig + @ComponentScan
- @RestController + @RequestMapping build REST APIs
- @ControllerAdvice for global exception handling
- @Entity + @Repository (JpaRepository) for database
- application.properties for configuration
- @EnableWebSecurity for security setup
- BCryptPasswordEncoder for password hashing
- JWT often used for stateless authentication

MODULE 6: MICROSERVICES

6.1 Principles

Microservices: architectural style structuring application as collection of small, independent services communicating over well-defined APIs.

- Service Independence: each service runs in its own process, deploy independently.
- Business Capability: each service handles a specific business function.
- Resilience & Scalability: services scale independently, failures isolated.

6.2 Service Discovery (Eureka)

```
@SpringBootApplication @EnableEurekaServer
public class EurekaServer { }

// application.yml
server.port=8761
eureka.client.register-with-eureka=false

// Client:
@SpringBootApplication @EnableDiscoveryClient
public class OrderService { }

spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

6.3 API Gateway

```
@SpringBootApplication @EnableZuulProxy
public class ApiGateway { }

# application.yml
zuul:
routes:
order-service: /api/orders/**
payment-service: /api/payments/**
```

6.4 Inter-Service Communication

Synchronous (Feign Client)

```
@FeignClient(name="payment-service")
public interface PaymentClient {
@PostMapping("/api/payments")
PaymentResult processPayment(@RequestBody PaymentRequest req);
}
```

Asynchronous (Kafka/Messaging)

```
@Service
public class OrderEventPublisher {
@Autowired private KafkaTemplate kafka;
public void publish(Order o) {
kafka.send("order-events", new OrderEvent(o.getId()));
}
}
```

6.5 Resilience Patterns

```
// Circuit Breaker
@Bean
public CircuitBreakerFactory circuitBreakerFactory() { return new Resilience4JCircuitBreakerFactory(); }

// @Retryable
@Retryable(value=ServiceUnavailableException.class, maxAttempts=3, backoff=@Backoff(delay=1000))
public Order createOrder(OrderRequest req) { }
```

Quick Revision

- Microservices: small, independent, business-focused services
- Eureka: service discovery and registration
- API Gateway: single entry point, routing, filtering
- Feign: declarative REST client for inter-service calls
- Kafka: async messaging for decoupled communication
- Circuit Breaker: prevents cascading failures
- @Retryable: automatic retry on failure
- Distributed tracing: track requests across services

MODULE 7: DESIGN PATTERNS

7.1 Creational Patterns

Singleton

```
public class DBConnection {
private static volatile DBConnection instance;
private DBConnection() { }
public static DBConnection getInstance() {
if (instance == null) {
synchronized(DBConnection.class) {
if (instance == null) instance = new DBConnection();
}
}
return instance;
}
}
```

Factory

```
interface PaymentProcessor { void process(double amt); }
class CreditCardProcessor implements PaymentProcessor { }
class PaymentFactory {
PaymentProcessor create(String type) {
return switch(type) {
case "credit" -> new CreditCardProcessor();
default -> throw new IllegalArgumentException();
};
}
}
```

Builder

```
User user = new User.Builder()
.username("john")
.email("john@x.com")
.firstName("John")
.build();
```

7.2 Structural Patterns

Adapter

```
interface PaymentGateway { void pay(double amt); }
class LegacySystem { void makePayment(String cur, double amt) { } }
class Adapter implements PaymentGateway {
private LegacySystem legacy;
public void pay(double amt) { legacy.makePayment("USD", amt); }
}
```

Decorator

```
interface Coffee { double cost(); }
class SimpleCoffee implements Coffee { public double cost() { return 1.0; } }
class MilkDecorator implements Coffee {
private Coffee coffee;
public MilkDecorator(Coffee c) { this.coffee = c; }
public double cost() { return coffee.cost() + 0.5; }
}
```

7.3 Behavioral Patterns

Observer

```
interface Observer { void update(String msg); }
class NewsAgency {
```

```
private List observers = new ArrayList<>();
void publish(String news) { observers.forEach(o -> o.update(news)); }
}
```

Strategy

```
interface SortStrategy { void sort(List list); }
class Sorter {
private SortStrategy strategy;
void sort(List l) { strategy.sort(l); }
}
```

Quick Revision

- Singleton: private constructor + static getInstance + double-check locking
- Factory: interface for creating objects, subclasses decide which class
- Builder: constructs complex objects step-by-step
- Adapter: makes incompatible interfaces compatible
- Decorator: adds behavior dynamically (wraps object)
- Observer: one-to-many notification on state change
- Strategy: family of algorithms, interchangeable at runtime
- Spring uses DI/Template method patterns extensively

CROSS-CONCEPT COMPARISON TABLES

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal	Resizable array	Doubly-linked list
get(i)	O(1)	O(n)
add/remove end	O(1) amortized	O(1)
add/remove middle	O(n) shift	O(1) ref change
Memory	Less	More (prev/next pointers)

HashMap vs TreeMap vs LinkedHashMap

Feature	HashMap	TreeMap	LinkedHashMap
Order	None	Sorted by key	Insertion order
Null key	Yes	No	Yes
Performance	O(1)	O(log n)	O(1)
Impl	Hash table	Red-Black tree	Hash+LinkedList

Abstract Class vs Interface

Feature	Abstract Class	Interface
Constructors?	Yes	No
Fields	Any	public static final
Methods	abstract+concrete	abstract+default+static
Multiple?	No (single)	Yes (multi-implement)
When	Shared state+behavior	Contract/capability

Checked vs Unchecked Exceptions

Aspect	Checked	Unchecked
Check time	Compile-time	Runtime
Must handle?	Yes (try-catch or throws)	Not required
Extends	Exception	RuntimeException
Examples	IOException, SQLException	NPE, ArithmeticException, ArrayIndexOutOfBounds

String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable?	No (immutable)	Yes	Yes
Thread-safe?	Yes (inherently)	No	Yes (synchronized)
Performance	Slow for modifications	Fastest	Slightly slower
When	Fixed strings, constants	Single-threaded many changes	Multi-threaded many changes

Runnable vs Callable

Feature	Runnable	Callable
Method	void run()	V call()
Return	No	Yes (Future.get())
Exceptions	Cannot throw checked	Can throw checked
Executed by	Thread	ExecutorService

@Component vs @Service vs @Repository vs @Controller

Feature	@Component	@Service	@Repository	@Controller
Generic component	Y	N	N	N
Business logic	N	Y	N	N
DAO/Persistence	N	N	Y	N
Web/MVC	N	N	N	Y
Exception translation	N	N	Y	N

END-OF-MODULE MOCK INTERVIEW

30 mixed questions covering all modules, ordered Easy -> Medium -> Hard.

EASY 1: What is JDK?

Answer: Java Development Kit = JRE + compilers, debuggers, tools for developing Java apps.

EASY 2: What is a constructor?

Answer: Special method with class name, no return type. Initializes object state.

EASY 3: Difference between == and .equals()?

Answer: == compares references. .equals() compares values/contents.

EASY 4: What is an array?

Answer: Fixed-size, 0-indexed container for elements of same type. length = property.

EASY 5: What is the default package import?

Answer: java.lang.* (imported automatically: String, Math, System, Object, etc.).

EASY 6: What is @Override?

Answer: Annotation indicating method overrides superclass method. Catches signature errors.

EASY 7: What is try-catch-finally?

Answer: Error handling: try has risky code, catch handles, finally always executes.

EASY 8: What is @FunctionalInterface?

Answer: Interface with exactly one abstract method. Used for lambda expressions.

EASY 9: What is @SpringBootApplication?

Answer: Combines @Configuration, @EnableAutoConfiguration, @ComponentScan.

EASY 10: What is the difference between @RestController and @Controller?

Answer: @RestController = @Controller + @ResponseBody. Returns data directly, not views.

MEDIUM 11: How does HashMap work?

Answer: Array of Node buckets. put(): hashCode() to find bucket -> equals() to check key. Java 8+: tree if >8 nodes. O(1) average.

MEDIUM 12: What is method hiding?

Answer: Static methods in subclass hide (not override) parent static methods. Call depends on reference type, not object type.

MEDIUM 13: What is the diamond problem in Java?

Answer: Two interfaces define same default method. Java resolves: class > sub-interface > must override explicitly.

MEDIUM 14: Explain Spring AOP proxy mechanism.

Answer: Spring uses JDK dynamic proxy (interfaces) or CGLIB (classes). Aspects woven at runtime. Pointcut+Advice = Aspect.

MEDIUM 15: What is @Transactional propagation REQUIRES_NEW?

Answer: Suspends current transaction (if any) and creates new one. Independent commit/rollback.

MEDIUM 16: What is fail-fast vs fail-safe?

Answer: Fail-fast: throws ConcurrentModificationException (ArrayList). Fail-safe: works on clone (ConcurrentHashMap).

MEDIUM 17: How to make a class immutable?

Answer: Declare final, private final fields, no setters, return copies of mutable objects in getters, only constructor initialization.

MEDIUM 18: What is volatile?

Answer: Ensures variable changes are visible across threads. Prevents thread-local caching. Does NOT provide atomicity.

MEDIUM 19: What is Stream pipeline?

Answer: Source -> zero+ intermediate ops (lazy, return Stream) -> terminal op (eager, triggers execution).

MEDIUM 20: Difference between @Component, @Service, @Repository?

Answer: All stereotypes. @Service: business logic. @Repository: DAO, translates SQL exceptions. @Controller: web layer.

HARD 21: Design a thread-safe Singleton with lazy initialization in Java.

Answer: Bill Pugh pattern: private static class Holder { static final INSTANCE = new Singleton(); }. JVM ensures thread safety on class loading.

HARD 22: How does ConcurrentHashMap achieve thread-safety?

Answer: Java 7: Segment-based locking (16 locks). Java 8+: CAS + synchronized on individual buckets. No lock on reads.

HARD 23: Explain the equals() and hashCode() contract with HashMap behavior.

Answer: equals() true -> hashCode() MUST equal. HashMap uses hashCode() to find bucket, then equals() to match key. Breaking contract causes incorrect lookups.

HARD 24: What is the difference between composition and inheritance, with a real example?

Answer: Inheritance: Dog extends Animal (tight coupling). Composition: Car has Engine (loose coupling). Favor composition.

HARD 25: How does Spring handle circular dependencies?

Answer: Constructor injection: throws BeanCurrentlyInCreationException. Setter/field injection: creates proxy, injects it, then resolves later. Use @Lazy for constructor cycles.

HARD 26: What is the difference between map() and flatMap()?

Answer: map: 1-to-1 transformation. flatMap: 1-to-N, flattens nested streams. Example: flatMap(line -> stream of words).

HARD 27: Explain JVM memory model (heap generations).

Answer: Young Gen (Eden + S0 + S1) -> Old Gen -> Metaspace. Minor GC: cleans Young. Major GC: cleans Old. Objects promoted after surviving multiple GC cycles.

HARD 28: What is the difference between synchronous and asynchronous microservice communication?

Answer: Sync (REST/Feign): caller blocks, tight coupling, simpler. Async (Kafka/RabbitMQ): non-blocking, loose coupling, eventually consistent, needs error handling.

HARD 29: Design a rate limiter for an API.

Answer: Token Bucket or Sliding Window algorithm. Tokens refill at fixed rate. Each request consumes one token. Reject if no tokens. Use Redis for distributed.

HARD 30: How does Spring Boot auto-configuration work?

Answer: @EnableAutoConfiguration loads spring.factories from META-INF. @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty control which configs activate. Example: if DataSource class on classpath, auto-configure DataSource bean.

FINAL CHEAT SHEET

Condensed 1-page visual summary for quick pre-interview revision.

Java Core

```
// 8 Primitives: byte short int long float double char boolean
// main: public static void main(String[] args)
// OOP: Encapsulation, Inheritance, Polymorphism, Abstraction
// String immutable, StringBuilder mutable
// == refs, .equals() values
// Checked exceptions: must handle. Unchecked: RuntimeException
// HashMap: O(1), hashCode()+equals()
// ArrayList: O(1) get; LinkedList: O(1) insert middle
// Generics: type safety at compile time
// Lambda: (params) -> expression
// Stream: filter/map (lazy) -> collect/reduce (eager)
// Thread: extends Thread OR implements Runnable
// synchronized: atomicity+visibility; volatile: visibility only
```

Spring Framework

```
// DI: constructor injection preferred
// @Component: generic; @Service: biz logic; @Repository: DAO; @Controller: web
// Scopes: singleton (default), prototype, request, session
// AOP: @Before, @After, @Around, @AfterReturning, @AfterThrowing
// @Transactional: declarative tx management

// Spring Boot:
// @SpringBootApplication = @Config + @EnableAuto + @ComponentScan
// @RestController = @Controller + @ResponseBody
// @Entity + @Repository (JpaRepository) = database
// @EnableWebSecurity + SecurityFilterChain = security
// application.properties: server.port, spring.datasource.*
```

Microservices

```
// Eureka: @EnableEurekaServer (server); @EnableDiscoveryClient (client)
// API Gateway: @EnableZuulProxy, routes config
// Feign: @FeignClient for declarative REST
// Kafka: async messaging
// Circuit Breaker: Resilience4J/Resilience4JCircuitBreakerFactory
// @Retryable: automatic retry
// Distributed Tracing: Sleuth + Zipkin
```

Design Patterns (in Spring)

```
// Singleton: @Scope("singleton") - default Spring beans
// Factory: @Bean methods
// Proxy: AOP proxies, @Lazy proxies
// Template: JdbcTemplate, RestTemplate, JmsTemplate
// Observer: ApplicationListener, @EventListener
// Strategy: ResourceLoader, AuthenticationProvider
// Decorator: HttpServletRequestWrapper, HttpServletResponseWrapper
```

KEY INTERVIEW TIPS

1. Always explain with examples
2. Compare with alternatives
3. Discuss trade-offs (performance, complexity, readability)
4. Mention real-world use cases
5. Be ready to code on paper/whiteboard
6. For Spring: explain what happens "under the hood"
7. For design: start with requirement, discuss options, then pick one

Prepared from Telusko's 63-Hour Complete Java Development Course by Navin Reddy
Java Full Stack Backend — Complete Study Notes